Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 1: Study and Empirical Analysis of Algorithms for Determining Fibonacci N-th Term

Author:
st. gr. FAF-242 Temciuc Adelina
Verified:
asist. univ. Fiștic Cristofor

Chișinău - 2025

Contents

ALGORITHM ANALYSIS **2**

Objective 2

Tasks 2

Theoretical Notes 2

Introduction 3

Comparison Metric 4

Input Format 4

IMPLEMENTATION **4**

Recursive Method 4

Dynamic Programming Method 6

Matrix Power Method 8

Binet Formula Method 11

Fast Doubling Method 13

CONCLUSION **17**

ALGORITHM ANALYSIS

Objective

Study and analyze different algorithms for determining Fibonacci n-th term.

Tasks:

1. Implement at least 3 algorithms for determining Fibonacci n-th term;
2. Decide properties of input format that will be used for algorithm analysis;
3. Decide the comparison metric for the algorithms;
4. Analyze empirically the algorithms;
5. Present the results of the obtained data;
6. Deduce conclusions of the laboratory.

Theoretical Notes:

An alternative to mathematical analysis of complexity is empirical analysis.

This may be useful for: obtaining preliminary information on the complexity class of an algorithm; comparing the efficiency of two (or more) algorithms for solving the same problems; comparing the efficiency of several implementations of the same algorithm; obtaining information on the efficiency of implementing an algorithm on a particular computer.

In the empirical analysis of an algorithm, the following steps are usually followed:

1. The purpose of the analysis is established.
2. Choose the efficiency metric to be used (number of executions of an operation (s) or time execution of all or part of the algorithm.
3. The properties of the input data in relation to which the analysis is performed are established (data size or specific properties).
4. The algorithm is implemented in a programming language.
5. Generating multiple sets of input data.
6. Run the program for each input data set.
7. The obtained data are analyzed.

The choice of the efficiency measure depends on the purpose of the analysis. If, for example, the aim is to obtain information on the complexity class or even checking the

accuracy of a theoretical estimate then it is appropriate to use the number of operations performed. But if the goal is to assess the behavior of the implementation of an algorithm then execution time is appropriate.

After the execution of the program with the test data, the results are recorded and, for the purpose of the analysis, either synthetic quantities (mean, standard deviation, etc.) are calculated or a graph with appropriate pairs of points (i.e. problem size, efficiency measure) is plotted.

Introduction:

The Fibonacci sequence is the series of numbers where each number is the sum of the two preceding numbers. For example: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, ... Mathematically we can describe this as: xn= xn-1 + xn-2.

Many sources claim this sequence was first discovered or "invented" by Leonardo Fibonacci. The Italian mathematician, who was born around A.D. 1170, was initially known as Leonardo of Pisa. In the 19th century, historians came up with the nickname Fibonacci (roughly meaning "son of the Bonacci clan") to distinguish the mathematician from another famous Leonardo of Pisa. There are others who say he did not. Keith Devlin, the author of Finding Fibonacci: The Quest to Rediscover the Forgotten Mathematical Genius Who Changed the World, says there are ancient Sanskrit texts that use the Hindu-Arabic numeral system - predating Leonardo of Pisa by centuries. But, in 1202 Leonardo of Pisa published a mathematical text, Liber Abaci. It was a "cookbook" written for tradespeople on how to do calculations. The text laid out the Hindu-Arabic arithmetic useful for tracking profits, losses, remaining loan balances, etc, introducing the Fibonacci sequence to the Western world.

Traditionally, the sequence was determined just by adding two predecessors to obtain a new number, however, with the evolution of computer science and algorithmics, several distinct methods for determination have been uncovered. The methods can be grouped in 4 categories, Recursive Methods, Dynamic Programming Methods, Matrix Power Methods, and Benet Formula Methods. All those can be implemented naively or with a certain degree of optimization, that boosts their performance during analysis.

As mentioned previously, the performance of an algorithm can be analyzed mathematically (derived through mathematical reasoning) or empirically (based on experimental observations).

Within this laboratory, we will be analyzing the 4 naïve algorithms empirically.

Comparison Metric:

The comparison metric for this laboratory work will be considered the time of execution of each algorithm (T(n))

Input Format:

As input, each algorithm will receive two series of numbers that will contain the order of the Fibonacci terms being looked up. The first series will have a more limited scope, (5, 7, 10, 12, 15, 17, 20, 22, 25, 27, 30, 32, 35, 37, 40, 42, 45), to accommodate the recursive method, while the second series will have a bigger scope to be able to compare the other algorithms between themselves (501, 631, 794, 1000, 1259, 1585, 1995, 2512, 3162, 3981, 5012, 6310, 7943, 10000, 12589, 15849).

IMPLEMENTATION

All four algorithms will be implemented in their naïve form in python an analyzed empirically based on the time required for their completion. While the general trend of the results may be similar to other experimental observations, the particular efficiency in rapport with input will vary depending o memory of the device used.

The error margin determined will constitute 2.5 seconds as per experimental measurement.

Recursive Method:

The recursive method, also considered the most inefficient method, follows a straightforward approach of computing the n-th term by computing it's predecessors first, and then adding them. However, the method does it by calling upon itself a number of times and repeating the same operation, for the same term, at least twice, occupying additional memory and, in theory, doubling it's execution time.

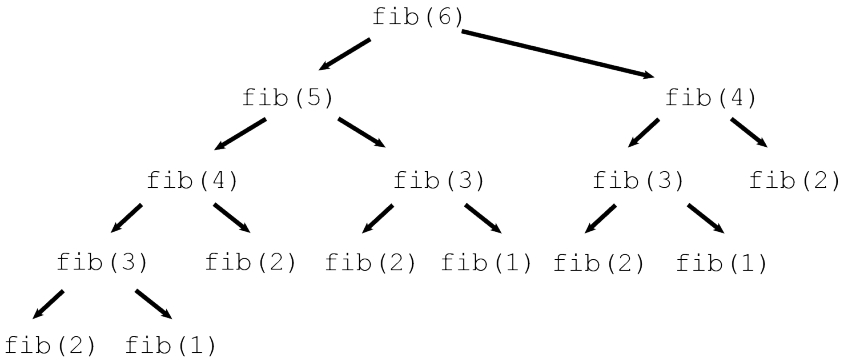

Figure 1: Fibonacci Recursion

Algorithm Description:

The naïve recursive Fibonacci method follows the algorithm as shown in the next pseudocode:

Listing 1: Recursive Fibonacci (pseudocode)

```
Fibonacci(n):
    if n <= 1:
        return n
    otherwise:
        return Fibonacci(n-1) + Fibonacci(n-2)
```

Implementation:

```python
def nth_fibonacci(n):
    if n <= 1:
        return n

    return nth_fibonacci(n-1) + nth_fibonacci(n-2)
```

Figure 2: Fibonacci recursion in Python

Results:

After running the function for each n Fibonacci term proposed in the list from the first Input Format and saving the time for each n, we obtained the following results:

n	run1(ms)	run2(ms)	run3(ms)	avg(ms)
5	0.002	0.001	0.001	0.001
7	0.002	0.002	0.002	0.002
10	0.007	0.007	0.007	0.007
12	0.020	0.019	0.019	0.019
15	0.078	0.072	0.079	0.076
17	0.206	0.185	0.185	0.192
20	0.795	0.795	0.795	0.795
22	2.084	2.101	2.111	2.098
25	10.802	9.871	9.671	10.115
27	24.701	24.745	24.241	24.563
30	102.679	130.023	102.340	111.681
32	268.811	269.037	273.597	270.482
35	1128.826	1163.213	1148.777	1146.939
37	3180.844	3072.293	5064.227	3772.455
40	17955.552	13385.507	13495.929	14945.663
42	40069.629	48487.447	48376.886	45644.654
45	202328.213	200205.282	209616.202	204049.899

Figure 3: Results for first set of inputs

In Figure 3 is represented the table of results for the first set of inputs. The highest line(the name of the columns) denotes the Fibonacci n-th term for which the functions

were run. Starting from the second row, we get the number of seconds that elapsed from when the function was run till when the function was executed. We may notice that the only function whose time was growing for this few n terms was the Recursive Method Fibonacci function.

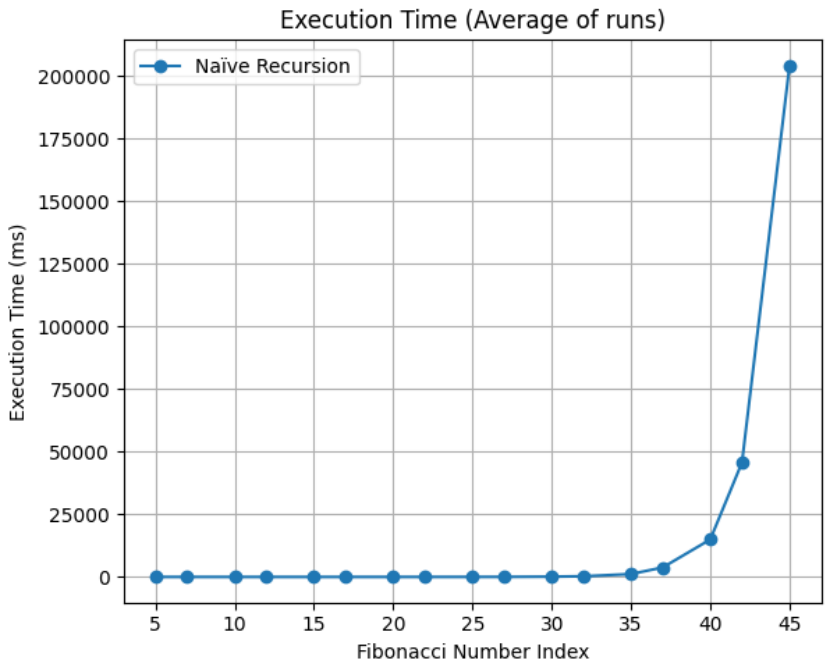

Figure 4: Graph of Recursive Fibonacci Function

Not only that, but also in the graph in Figure 4 that shows the growth of the time needed for the operations, we may easily see the spike in time complexity that happens after the 42nd term, leading us to deduce that the Time Complexity is exponential. $T(2^n)$.

Dynamic Programming Method:

The Dynamic Programming method, similar to the recursive method, takes the straightforward approach of calculating the n-th term. However, instead of calling the function upon itself, from top down it operates based on an array data structure that holds the previously computed terms, eliminating the need to recompute them.

Algorithm Description:

The naïve DP algorithm for Fibonacci n-th term follows the pseudocode:

Listing 2: Dynamic Programming Method (pseudocode)

```
Fibonacci(n):
    Array A;
    A[0]<-0;
    A[1]<-1;
```

```
    for i <- 2 to  n 1  do
        A[i]<-A[i-1]+A[i-2];
    return A[n-1]
```

Implementation:

```python
def nthFibonacci(n):

    if n <= 1:
        return n

    dp = [0] * (n + 1)

    dp[0], dp[1] = 0, 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

Figure 5: Fibonacci DP in Python

Results:

After the execution of the function for each n Fibonacci term mentioned in the second set of Input Format we obtain the following results:

n	run1(ms)	run2(ms)	run3(ms)	run4(ms)	run5(ms)	avg(ms)
501	0.051	0.035	0.042	0.052	0.044	0.045
631	0.058	0.042	0.038	0.038	0.037	0.043
794	0.059	0.050	0.048	0.048	0.048	0.051
1000	0.075	0.066	0.065	0.065	0.064	0.067
1259	0.116	0.103	0.104	0.100	0.095	0.104
1585	0.129	0.115	0.118	0.111	0.110	0.116
1995	0.172	0.152	0.148	0.147	0.147	0.153
2512	0.238	0.197	0.194	0.194	0.194	0.203
3162	0.321	0.275	0.262	0.262	0.261	0.276
3981	0.421	0.366	0.361	0.359	0.365	0.374
5012	0.675	0.718	0.730	0.723	0.722	0.714
6310	1.191	1.101	1.509	1.825	2.428	1.611
7943	2.865	2.382	2.194	2.003	2.095	2.308
10000	6.013	3.727	3.500	3.359	5.851	4.490
12589	5.998	4.355	9.025	5.024	4.830	5.846
15849	10.550	10.006	8.103	10.062	7.294	9.203

Figure 6: Fibonacci DP Results

With the Dynamic Programming Method (first row, row[0]) showing excellent results with a time complexity denoted in a corresponding graph of T(n),

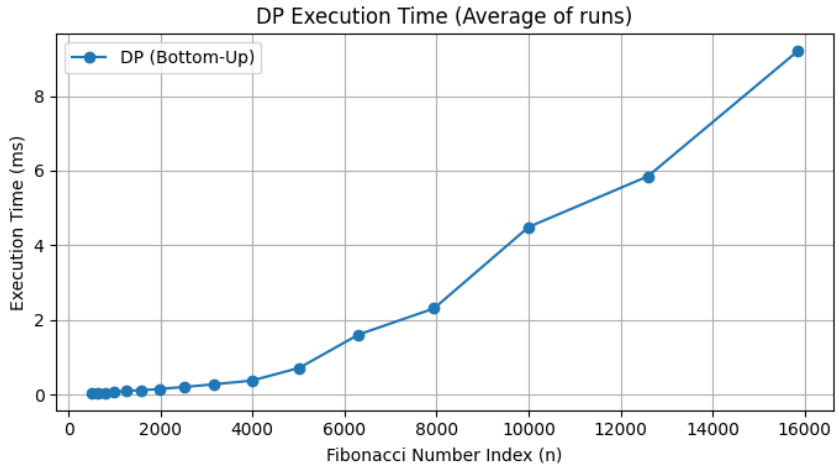

Figure 7: Fibonacci DP Graph

Not only that, but also in the graph in Figure 4 that shows the growth of the time needed for the operations, we may easily see the spike in time complexity that happens after the 42nd term, leading us to deduce that the Time Complexity is exponential. T(2^n).

Matrix Power Method:

The Matrix Power method of determining the n-th Fibonacci number is based on, as expected, the multiple multiplication of a naïve Matrix $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$ with itself.

Algorithm Description:

It is known that

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} b \\ a+b \end{pmatrix}$$

This property of Matrix multiplication can be used to represent

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_1 \\ F_2 \end{pmatrix}$$

And similarly:

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^2 \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_2 \\ F_3 \end{pmatrix}$$

Which turns into the general:

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}$$

This set of operation can be described in pseudocode as follows:

Listing 3: Matrix Power Method (pseudocode)

```
Fibonacci(n):
    F<- []
    vec <- [[0], [1]]
    Matrix <- [[0, 1],[1, 1]]
    F <-power(Matrix, n)
    F <- F * vec
    Return F[0][0]
```

Implementation: The implementation of the driving function in Python is as follows:

```python
def nthFibonacci(n):
    if n <= 1:
        return n

    # Initialize the transformation matrix
    mat1 = [[1, 1], [1, 0]]

    matrixPower(mat1, n - 1)

    # The result is in the top-left cell of the matrix
    return mat1[0][0]
```

Figure 8: Fibonacci Matrix Power Method in Python

With additional miscellaneous functions:

```python
def matrixPower(mat1, n):

    # Base case for recursion
    if n == 0 or n == 1:
        return

    mat2 = [[1, 1], [1, 0]]

    matrixPower(mat1, n // 2)

    multiply(mat1, mat1)

    # If n is odd, multiply by the helper matrix mat2
    if n % 2 != 0:
        multiply(mat1, mat2)
```

Figure 9: Power Function Python

Where the power function (Figure 8) handles the part of raising the Matrix to the power n, while the multiplying function (Figure 9) handles the matrix multiplication with itself.

```python
def multiply(mat1, mat2):
    x = mat1[0][0] * mat2[0][0] + mat1[0][1] * mat2[1][0]
    y = mat1[0][0] * mat2[0][1] + mat1[0][1] * mat2[1][1]
    z = mat1[1][0] * mat2[0][0] + mat1[1][1] * mat2[1][0]
    w = mat1[1][0] * mat2[0][1] + mat1[1][1] * mat2[1][1]

    # Update matrix mat1 with the result
    mat1[0][0], mat1[0][1] = x, y
    mat1[1][0], mat1[1][1] = z, w
```

Figure 10: Multiply Function Python

Result : After the execution of the function for each n Fibonacci term mentioned in the second set of Input Format we obtain the following results:

n	run1(ms)	run2(ms)	run3(ms)	run4(ms)	run5(ms)	avg(ms)
501	0.016	0.009	0.008	0.008	0.007	0.010
631	0.029	0.011	0.010	0.009	0.010	0.014
794	0.026	0.012	0.011	0.011	0.009	0.014
1000	0.015	0.013	0.013	0.010	0.010	0.012
1259	0.014	0.013	0.011	0.011	0.011	0.012
1585	0.015	0.014	0.014	0.012	0.012	0.013
1995	0.020	0.019	0.015	0.015	0.015	0.017
2512	0.025	0.020	0.020	0.020	0.019	0.021
3162	0.029	0.024	0.023	0.023	0.023	0.024
3981	0.039	0.030	0.030	0.030	0.030	0.032
5012	0.049	0.040	0.040	0.040	0.040	0.042
6310	0.065	0.056	0.055	0.055	0.054	0.057
7943	0.079	0.073	0.073	0.073	0.072	0.074
10000	0.117	0.110	0.110	0.109	0.109	0.111
12589	0.152	0.144	0.141	0.141	0.141	0.144
15849	0.217	0.210	0.210	0.209	0.209	0.211

Figure 11: Matrix Method Fibonacci Results

With the naïve Matrix method (indicated in last row, row[2]), although being slower than the Binet and Dynamic Programming one, still performing pretty well, with the form f the graph indicating a pretty solid T(n) time complexity.

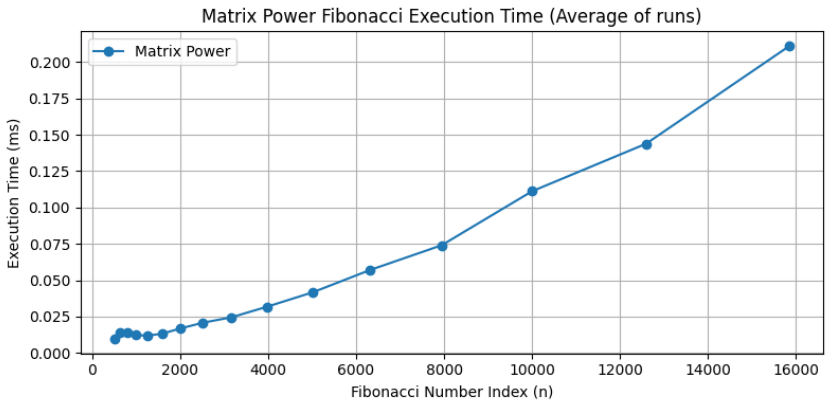

Figure 12: Matrix Method Fibonacci graph

Binet Formula Method:

The Binet Formula Method is another unconventional way of calculating the n-th term of the Fibonacci series, as it operates using the Golden Ratio formula, or phi. However, due to its nature of requiring the usage of decimal numbers, at some point, the rounding error of python that accumulates, begins affecting the results significantly. The observation of error starting with around 70-th number making it unusable in practice, despite its speed.

Algorithm Description:

The set of operation for the Binet Formula Method can be described in pseudocode as follows:

Listing 4: Binet Method (pseudocode)

```
Fibonacci(n):
    phi <- (1 + sqrt(5))
    phi1 <-(1      sqrt(5))
    return pow(phi, n)- pow(phi1, n)/(pow(2, n)*sqrt(5))
```

Implementation: The implementation of the function in Python is as follows, with some alterations that would increase the number of terms that could be obtain through it:

```python
def fibonacci(n):

    phi = (1 + math.sqrt(5)) / 2
    psi = (1 - math.sqrt(5)) / 2

    return round((math.pow(phi, n) - math.pow(psi, n)) / math.sqrt(5))
```

Figure 13: Fibonacci Binet Formula Method in Python

Result : Although the most performant with its time, as shown in the table of results, in row [1],

n	F(n)	run1(ms)	run2(ms)	run3(ms)	run4(ms)	run5(ms)	avg(ms)
5	5	0.012200	0.003000	0.000900	0.000700	0.000500	0.003460
7	13	0.002700	0.000700	0.000600	0.000600	0.000600	0.001040
10	55	0.004200	0.001000	0.000600	0.000600	0.000500	0.001380
12	144	0.002400	0.000700	0.000600	0.000500	0.000600	0.000960
15	610	0.002900	0.000800	0.000600	0.000500	0.000600	0.001080
17	1597	0.002900	0.000900	0.000600	0.000500	0.000600	0.001100
20	6765	0.002800	0.000800	0.000600	0.000500	0.000600	0.001060
22	17711	0.002400	0.000700	0.000600	0.000600	0.000600	0.000980
25	75025	0.002400	0.000700	0.000700	0.000500	0.000600	0.000980
27	196418	0.002800	0.000800	0.000700	0.000500	0.000600	0.001080
30	832040	0.002300	0.000800	0.000600	0.000600	0.000600	0.000980
32	2178309	0.002300	0.000700	0.000800	0.000600	0.000600	0.001000
35	9227465	0.002700	0.000900	0.000600	0.000600	0.000700	0.001100
37	24157817	0.002700	0.000900	0.000700	0.000700	0.000700	0.001140
40	102334155	0.002700	0.000900	0.000700	0.000600	0.000700	0.001120
42	267914296	0.001900	0.000800	0.000700	0.002900	0.013200	0.003900
45	1134903170	0.002800	0.001200	0.001000	0.001100	0.001000	0.001420

Figure 14: Fibonacci Binet Formula Method results

And as shown in its performance graph,

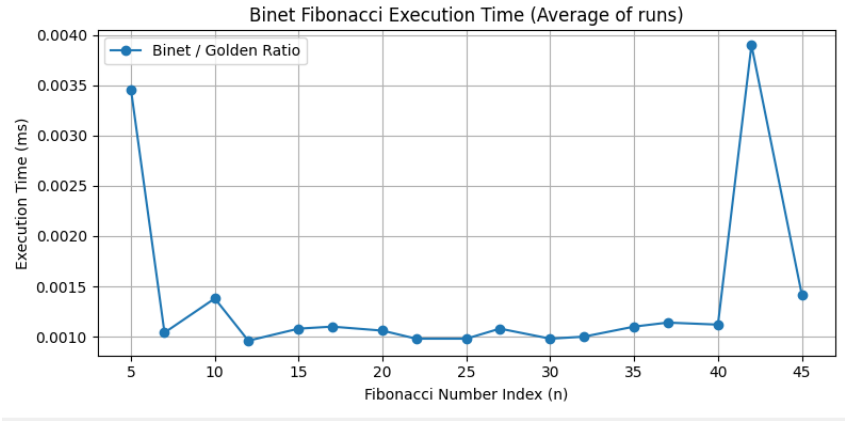

Figure 15: Fibonacci Binet formula Method

The Binet Formula Function is not accurate enough to be considered within the analysed limits and is recommended to be used for Fibonacci terms up to 80. At least in its naïve form in python, as further modification and change of language may extend its usability further.

Fast Doubling Method:

The Doubling Method can be seen as an improvement to the matrix exponentiation method to find the N-th Fibonacci number although it doesn't use matrix multiplication itself.

Algorithm Description:

Start with:

$$F(n+1) = F(n) + F(n-1), \qquad F(n) = F(n).$$

It can be rewritten in matrix form as:

$$\begin{bmatrix} F(n+1) \\ F(n) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^2 \begin{bmatrix} F(n-1) \\ F(n-2) \end{bmatrix} = \cdots = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} F(1) \\ F(0) \end{bmatrix}.$$

For doubling, we simply substitute $2n$ into the formula:

$$\begin{bmatrix} F(2n+1) \\ F(2n) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{2n} \begin{bmatrix} F(1) \\ F(0) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} F(1) \\ F(0) \end{bmatrix}.$$

Using the identity

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n = \begin{bmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{bmatrix},$$

we get:

$$\begin{bmatrix} F(2n+1) \\ F(2n) \end{bmatrix} = \begin{bmatrix} F(n+1)^2 + F(n)^2 \\ F(n)F(n+1) + F(n)F(n-1) \end{bmatrix}.$$

Substituting $F(n-1) = F(n+1) - F(n)$ and simplifying, we obtain:

$$\begin{bmatrix} F(2n+1) \\ F(2n) \end{bmatrix} = \begin{bmatrix} F(n+1)^2 + F(n)^2 \\ 2F(n+1)F(n) - F(n)^2 \end{bmatrix}.$$

Which gave us two basic formulas:

$$F(2n) = F(n)\big[2F(n+1) - F(n)\big],$$

$$F(2n+1) = F(n)^2 + F(n+1)^2.$$

This set of operation can be described in pseudocode as follows:

Listing 5: Fast Doubling Method (pseudocode)

```
Fibonacci(n):
    if n <= 1:
        return n

    (a, b) <- FastDoubling(n)      # a = F(n), b = F(n+1)
    return a


FastDoubling(n):
    if n == 0:
        return (0, 1)

    (a, b) <- FastDoubling(n // 2)      # a = F(k), b = F(k+1)

    c <- a * (2*b - a)                  # c = F(2k)
    d <- a*a + b*b                      # d = F(2k + 1)

    if n % 2 == 0:
        return (c, d)
    else:
        return (d, c + d)
```

Implementation: The implementation of the driving function in Python is as follows:

```python
def FastDoubling(n, res):
    if n == 0:
        res[0] = 0
        res[1] = 1
        return

    FastDoubling(n // 2, res)

    a = res[0]  # F(k)
    b = res[1]  # F(k+1)

    c = (2 * b - a) % MOD              # (2F(k+1) - F(k)) mod
    c = (a * c) % MOD                  # F(2k)
    d = (a * a + b * b) % MOD          # F(2k+1)

    if n % 2 == 0:
        res[0] = c
        res[1] = d
    else:
        res[0] = d
        res[1] = (c + d) % MOD         # IMPORTANT: mod here

def fib_fast_doubling_mod(n: int) -> int:
    res = [0, 0]
    FastDoubling(n, res)
    return res[0]
```

Figure 16: Fast Doubling Method in Python

Result : After the execution of the function for each n Fibonacci term mentioned in the second set of Input Format, we obtain the following results: the fast doubling method keeps an almost constant execution time (on the order of a few thousandths of a millisecond) even up to $n = 10^7$, with only minor fluctuations caused by measurement noise, which is consistent with its expected $O(\log n)$ complexity under modulo arithmetic.

n	F(n) mod 1e9+7	run1(ms)	run2(ms)	run3(ms)	run4(ms)	run5(ms)	run6(ms)	run7(ms)	run8(ms)	run9(ms)	run10(ms)	avg(ms)
501	0.009800	0.003800	0.002300	0.002000	0.002000	0.002100	0.002100	0.002000	0.001900	0.002000	0.003000
631	0.006500	0.004100	0.002800	0.002800	0.002400	0.002600	0.002500	0.002500	0.002400	0.002400	0.003100
794	0.006900	0.004100	0.003100	0.002700	0.002700	0.003000	0.002800	0.002600	0.002800	0.002600	0.003330
1000	0.006300	0.003900	0.002800	0.002600	0.002500	0.002500	0.002400	0.002400	0.002400	0.002400	0.003020
1259	0.007600	0.005600	0.003200	0.002800	0.002800	0.003000	0.002600	0.002800	0.002700	0.002800	0.003590
1585	0.007600	0.004900	0.003700	0.003400	0.003300	0.003500	0.003400	0.003300	0.003300	0.003300	0.003970
1995	0.008300	0.005700	0.004500	0.004100	0.004000	0.004100	0.004100	0.003800	0.003900	0.004100	0.004660
2512	0.009300	0.006600	0.005600	0.005000	0.005100	0.005300	0.005000	0.005000	0.004900	0.005000	0.005680
3162	0.010600	0.007900	0.006900	0.006500	0.006400	0.006400	0.006300	0.006200	0.006300	0.006200	0.006970
3981	0.016000	0.010900	0.009300	0.008900	0.009100	0.008800	0.008600	0.008600	0.008500	0.008600	0.009730
5012	0.015600	0.012600	0.011500	0.011000	0.011100	0.011100	0.010900	0.011000	0.010900	0.010900	0.011660
6310	0.022500	0.018400	0.016200	0.015800	0.015500	0.015900	0.015700	0.015700	0.015700	0.015600	0.016700
7943	0.027700	0.024000	0.023100	0.022600	0.022600	0.022100	0.022200	0.114600	0.033000	0.026100	0.033800
10000	0.042300	0.046000	0.037300	0.031400	0.031500	0.031600	0.035000	0.059500	0.036200	0.036200	0.038700
12589	0.060000	0.053000	0.051400	0.066500	0.050700	0.046300	0.043800	0.043600	0.050000	0.050300	0.051560
15849	0.083000	0.068700	0.067500	0.074500	0.069300	0.073600	0.073300	0.074100	0.066700	0.063900	0.071460

Figure 17: Fast Doubling Method Fibonacci Results

With the matrix exponentiation method (shown in the corresponding row of the table), although it is slower than the Binet and dynamic programming approaches for these inputs, it still performs efficiently. The execution-time curve grows smoothly and steadily, indicating a stable $T(n)$ behavior consistent with the expected logarithmic num-

ber of matrix multiplications, with the main cost coming from the increasing size of the intermediate Fibonacci values.

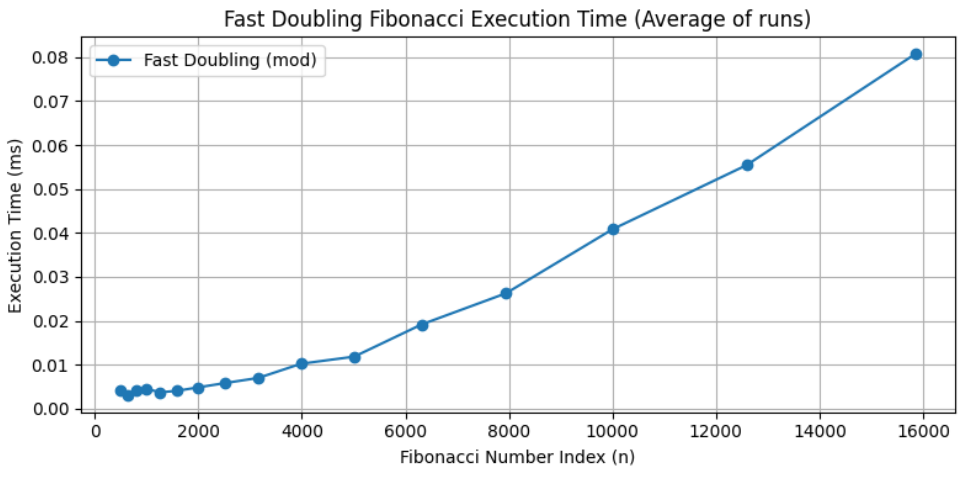

Figure 18: Fast Doubling Method Fibonacci graph

CONCLUSION

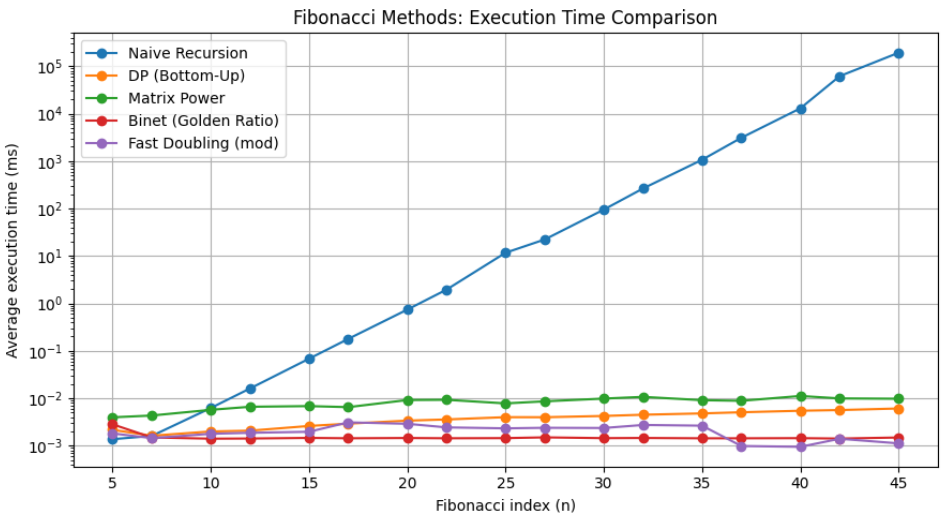

Figure 19: All methods Fibonacci graph

In conclusion, the execution-time comparison clearly shows how strongly the choice of algorithm affects computing the n-th Fibonacci term. Naive recursion becomes impractical very quickly: as n increases, its runtime grows explosively, dominating the chart by several orders of magnitude, which reflects its exponential time complexity caused by repeated recomputation of the same subproblems.

In contrast, the dynamic programming (bottom-up) approach remains consistently fast and scales smoothly because each Fibonacci value is computed once, giving linear time behavior. The matrix power method stays efficient as well, benefiting from logarithmic growth due to exponentiation by squaring, but it carries a slightly higher constant overhead than the simplest iterative DP. The closed-form (Binet/golden ratio) method appears extremely fast, yet it is best viewed as an approximation-based technique whose reliability depends on floating-point precision and can break for larger indices if exact integer results are required.

Finally, fast doubling (mod) provides the best overall performance and scalability on this plot: it maintains very low execution times while also producing exact results under modular arithmetic, making it the most suitable choice for large n when correctness and speed are both important.

Overall, the results confirm that while multiple methods can compute Fibonacci numbers efficiently, exponential recursion should be avoided, and logarithmic methods—especially fast doubling—offer the strongest performance for large inputs.

Github Repository: https://github.com/Adelina1905/Algorithm-Labs